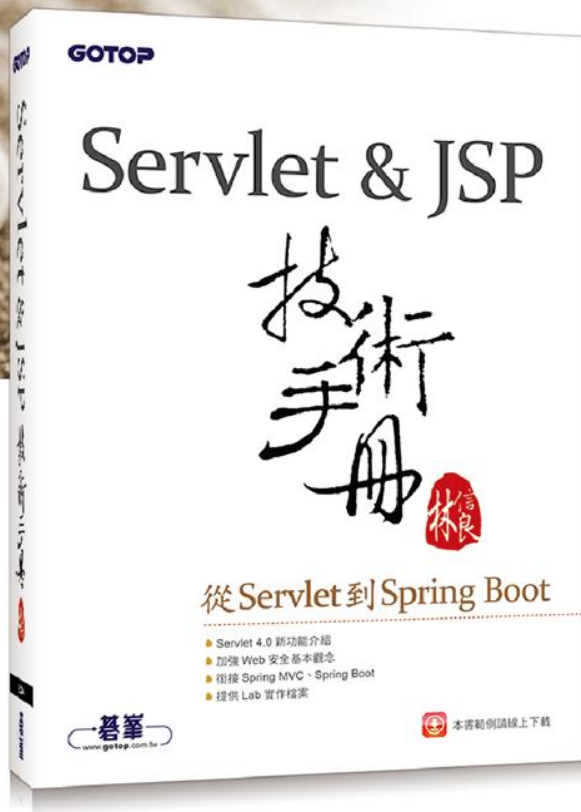




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 10

• Web容器安全管理



學習目標

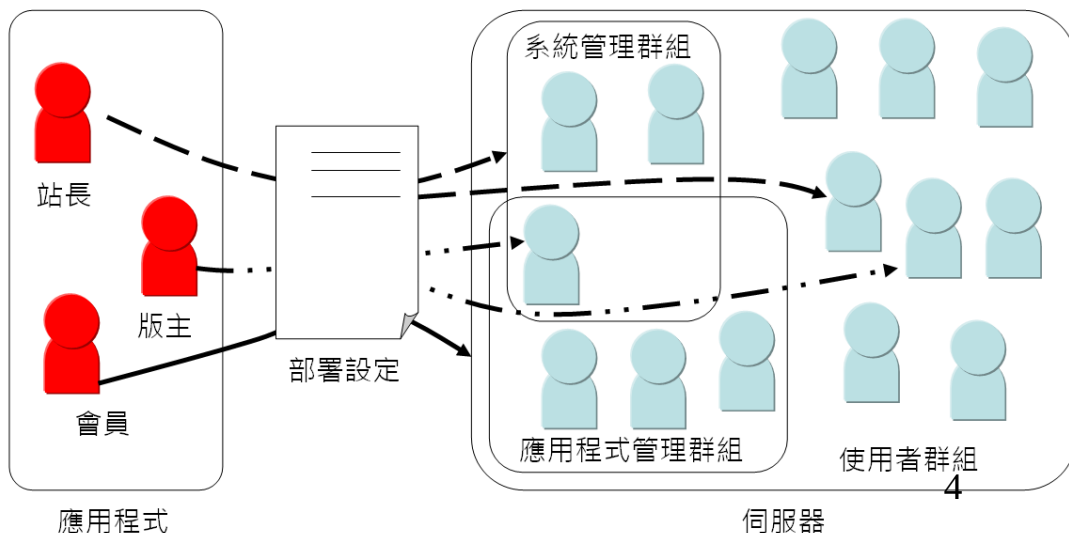
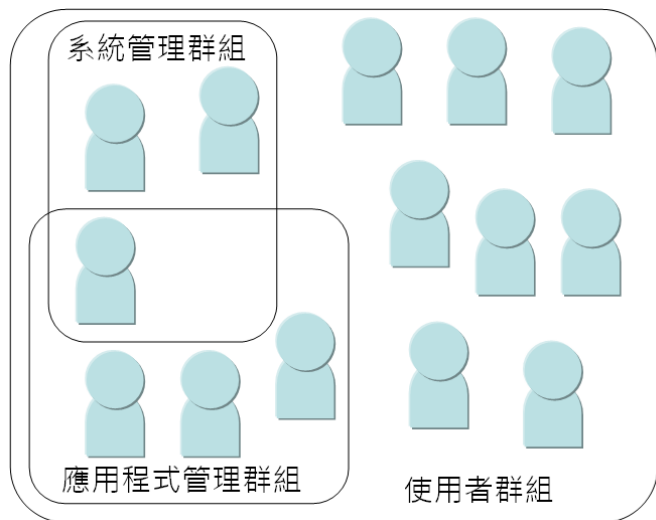
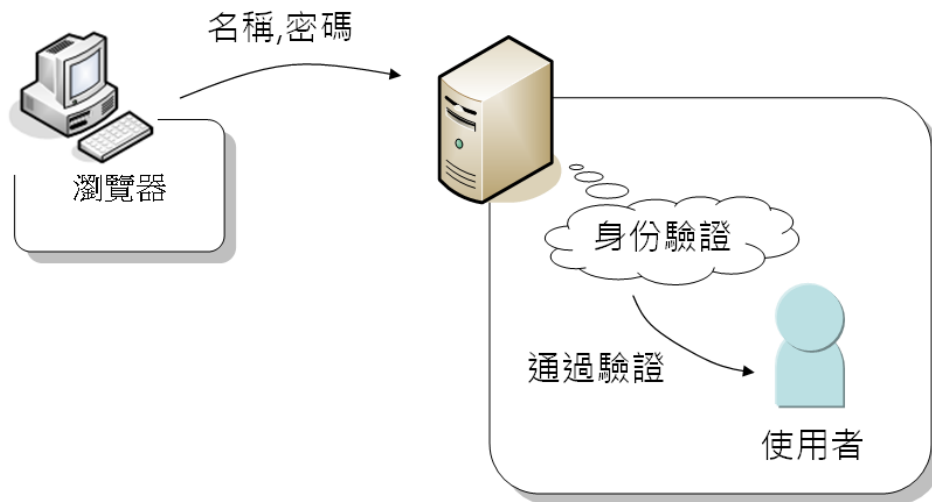
- 了解Java EE安全概念與名詞
- 使用容器基本驗證與表單驗證
- 使用HTTPS保密資料傳輸

Java EE安全基本觀念

- 驗證（Authentication）
- 資源存取控制（Access control for resources）
- 資料完整性（Data Integrity）
- 資料機密性或隱私性（Confidentiality or Data Privacy）

Java EE安全基本觀念

- 使用者 (User)
- 群組 (Group)
- 角色 (Role)
- Realm



Java EE安全基本觀念

- 宣告式安全（Declarative Security）
- 程設式安全（Programmatic Security）

宣告式基本驗證

需要進行驗證

http://localhost:8080

使用者名稱

密碼

登入

取消

宣告式基本驗證

- 讓Web容器提供基本驗證的功能，可以在web.xml中定義

```
<login-config>  
    <auth-method>BASIC</auth-method>  
</login-config>
```

- 定義角色

```
<security-role>  
    <role-name>admin</role-name>  
</security-role>  
<security-role>  
    <role-name>manager</role-name>  
</security-role>
```

宣告式基本驗證

- 定義哪些URL可以被哪些角色以哪種HTTP方法存取
- 看不到任何HTTP方法規範的定義，預設就是所有HTTP方法都受到限制

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Admin</web-resource-name>
    <url-pattern>/admin/*</url-pattern>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
  </auth-constraint>
</security-constraint>
```


宣告式基本驗證

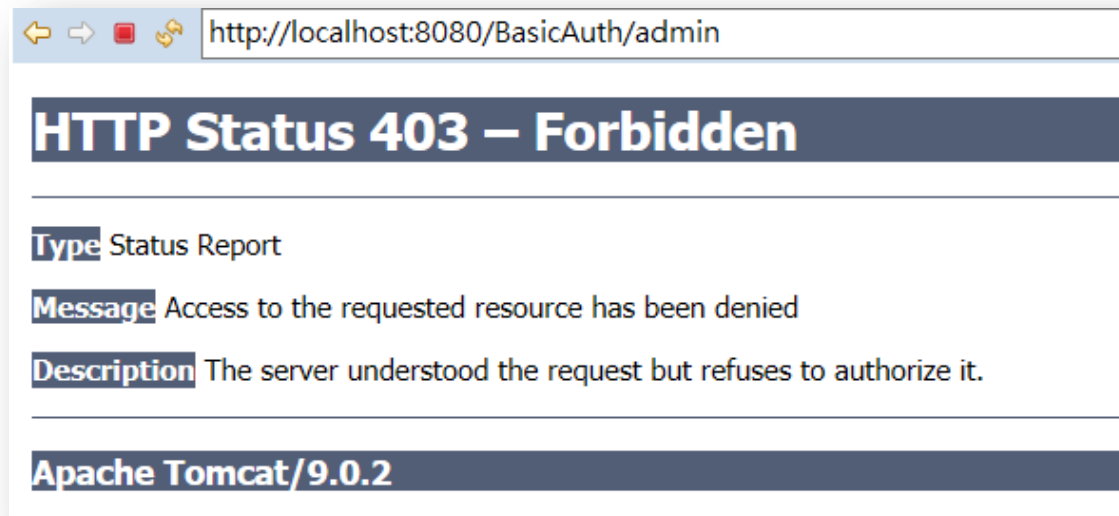
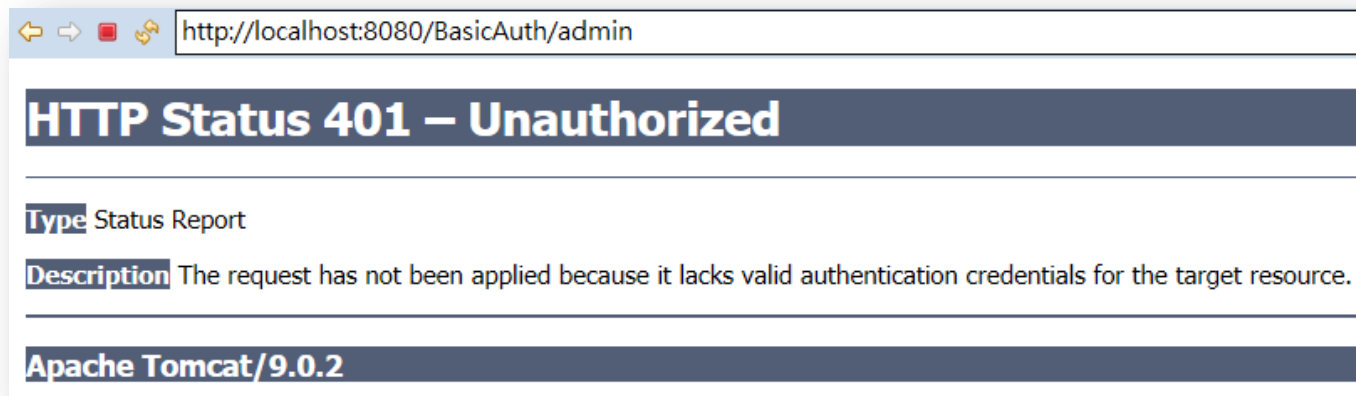
- 只有admin或manager才可以使用GET與POST方法進行存取

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Manager</web-resource-name>
    <url-pattern>/manager/*</url-pattern>
    <http-method>GET</http-method>
    <http-method>POST</http-method>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
    <role-name>manager</role-name>
  </auth-constraint>
</security-constraint>
```

宣告式基本驗證

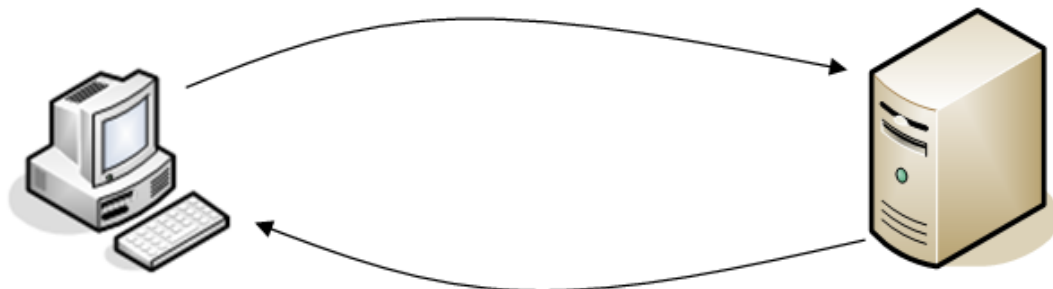
- 沒有設定`<http-method>`，則所有HTTP方法都會受到限制
- 設定了`<http-method>`，則只有被設定的HTTP方法受到限制，其它方法則不受限制
- 沒有設定`<auth-constraint>`標籤，或是`<auth-constraint>`標籤中設定`<role-name>*</role-name>`，表示任何角色都可以存取
- 直接撰寫`<auth-constraint/>`，那就沒有任何角色可以存取

宣告式基本驗證



容器基本驗證原理

Host localhost:8084
User-Agent Mozilla/5.0 (Windows; U; Windows NT 6.0; zh-TW; rv:1.9.0.10) Gecko/2009042316 Firefox/3.0.10 (.NET CLR 3.5.30729)
Accept text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language zh-tw,en-us;q=0.7,en;q=0.3
Accept-Encoding gzip,deflate
Accept-Charset Big5,utf-8;q=0.7,*;q=0.7
Keep-Alive 300
Connection keep-alive



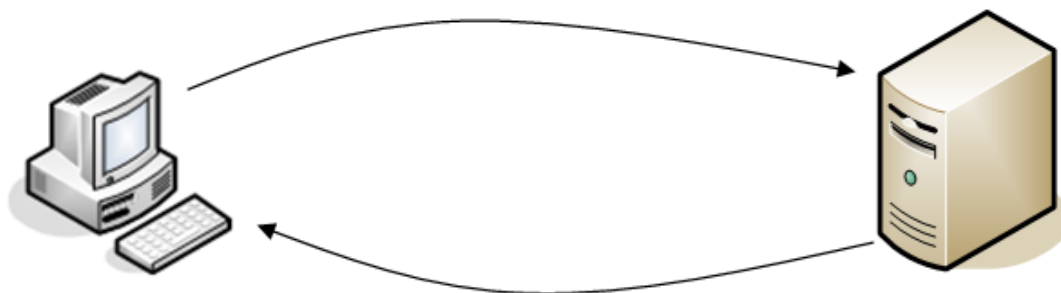
Server Apache-Coyote/1.1
Pragma No-cache
Cache-Control no-cache
Expires Thu, 01 Jan 1970 08:00:00 CST
WWW-Authenticate Basic realm="localhost:8084"
Content-Type text/html; charset=utf-8
Content-Length 954
Date Thu, 07 May 2009 01:55:46 GMT

WWW-Authenticate Basic realm="localhost:8084"

容器基本驗證原理

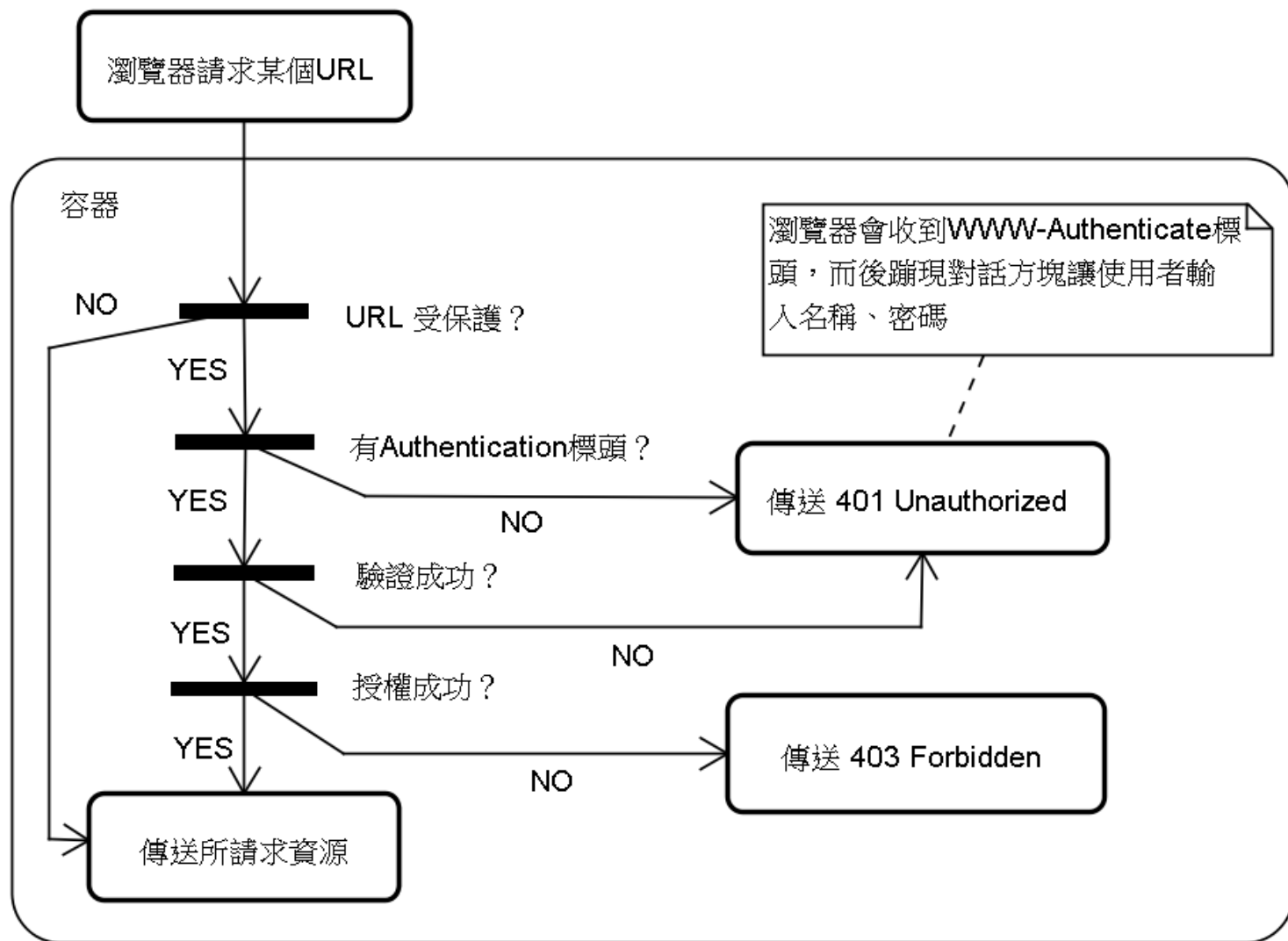
Host localhost:8084
User-Agent Mozilla/5.0 (Windows; U; Windows NT 6.0; zh-TW; rv:1.9.0.10) Gecko/2009042316 Firefox/3.0.10 (.NET CLR 3.5.30729)
Accept text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language zh-tw,en-us;q=0.7,en;q=0.3
Accept-Encoding gzip,deflate
Accept-Charset Big5,utf-8;q=0.7,*;q=0.7
Keep-Alive 300
Connection keep-alive
Authorization Basic Y2F0ZXJwaWxsYXI6MTIzNDU2

Authorization Basic Y2F0ZXJwaWxsYXI6MTIzNDU2



Server Apache-Coyote/1.1
Pragma No-cache
Cache-Control no-cache
Expires Thu, 01 Jan 1970 08:00:00 CST
Content-Type text/html; charset=utf-8
Content-Length 1024
Date Thu, 07 May 2009 02:15:10 GMT

容器基本驗證原理



宣告式表單驗證

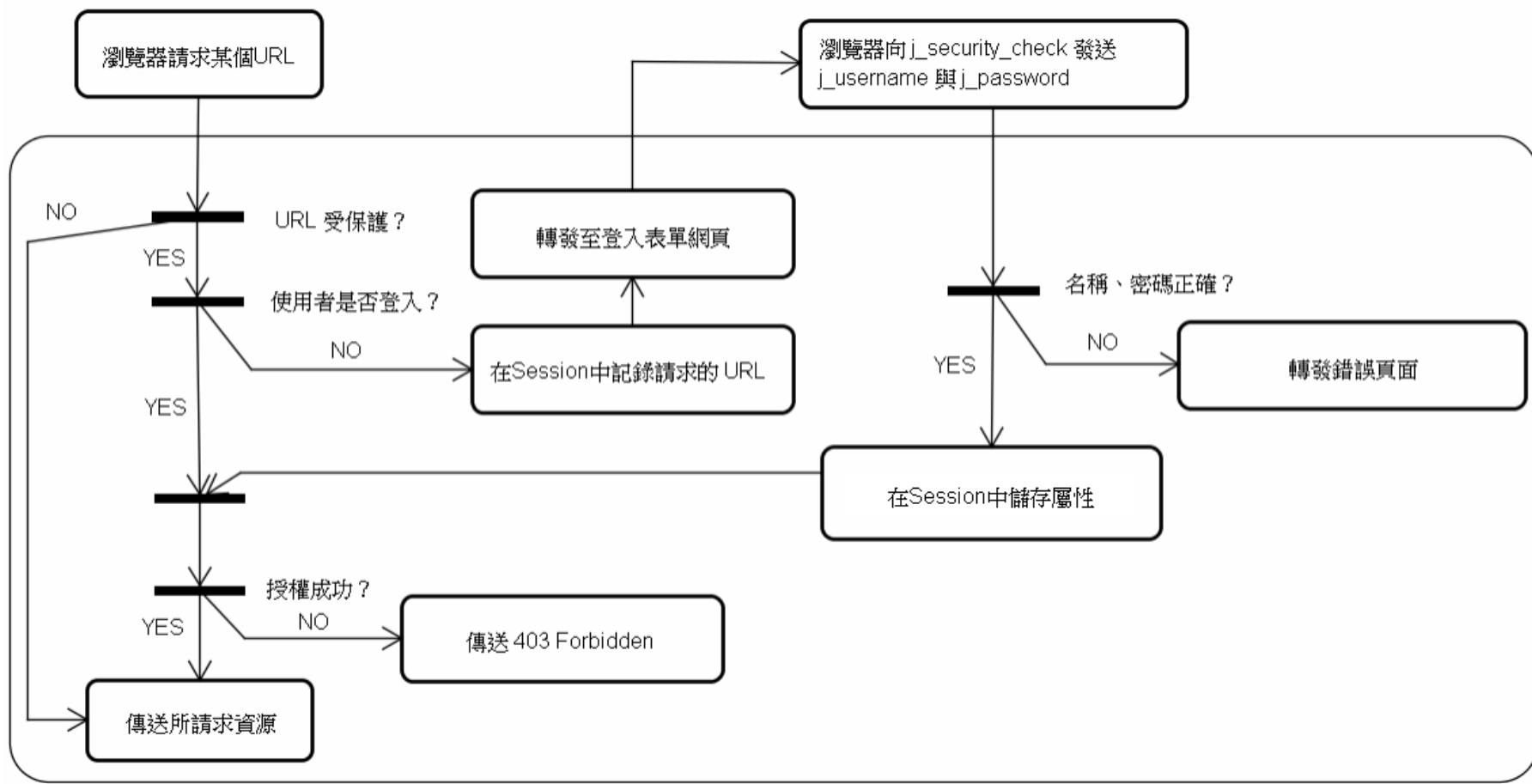
```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" version="3.1">

    // 略...
    <login-config>
        <auth-method>FORM</auth-method>
        <form-login-config>
            <form-login-page>/login.html</form-login-page>
            <form-error-page>/error.html</form-error-page>
        </form-login-config>
    </login-config>
    // 略...
</web-app>
```

宣告式表單驗證

```
<!DOCTYPE html>
<html>
  <head>
    <title>登入</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <form action="j_security_check" method="post">
      名稱:<input type="text" name="j_username"><br>
      密碼:<input type="password" name="j_password"><br>
      <input type="submit" value="送出">
    </form>
  </body>
</html>
```


容器表單驗證原理



使用HTTPS保護資料

```
<user-data-constraint>  
    <transport-guarantee>CONFIDENTIAL</transport-guarantee>  
</user-data-constraint>
```

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Admin</web-resource-name>
    <url-pattern>/admin/*</url-pattern>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
  </auth-constraint>
  <user-data-constraint>
    <transport-guarantee>CONFIDENTIAL</transport-guarantee>
  </user-data-constraint>
</security-constraint>
```

設定資料傳輸必須保證機密性與完整性

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Manager</web-resource-name>
    <url-pattern>/manager/*</url-pattern>
    <http-method>GET</http-method>
    <http-method>POST</http-method>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
    <role-name>manager</role-name>
  </auth-constraint>
  <user-data-constraint>
    <transport-guarantee>CONFIDENTIAL</transport-guarantee>
  </user-data-constraint>
</security-constraint>
```

設定資料傳輸必須保證機密性與完整性

程式安全管理

- 在Servlet 3.0中，HttpServletRequest新增了三個與安全有關的方法：
authenticate()、**login()**、**logout()**

程式安全管理

```
@WebServlet("/user")
public class User extends HttpServlet {
    @Override
    protected void doGet(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        if(request.authenticate(response)) {
            response.setContentType("text/html;charset=UTF-8");
            PrintWriter out = response.getWriter();
            out.println("必須驗證過使用者才可以看到的資料");
            out.println("<a href='logout'>登出</a>");
        }
    }
}
```

```
@WebServlet("/login")
public class Login extends HttpServlet {
    @Override
    protected void doPost(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String user = request.getParameter("user");
        String passwd = request.getParameter("passwd");
        try {
            request.login(user, passwd);
            response.sendRedirect("user");
        } catch (ServletException ex) {
            response.sendRedirect("login.html");
        }
    }
}

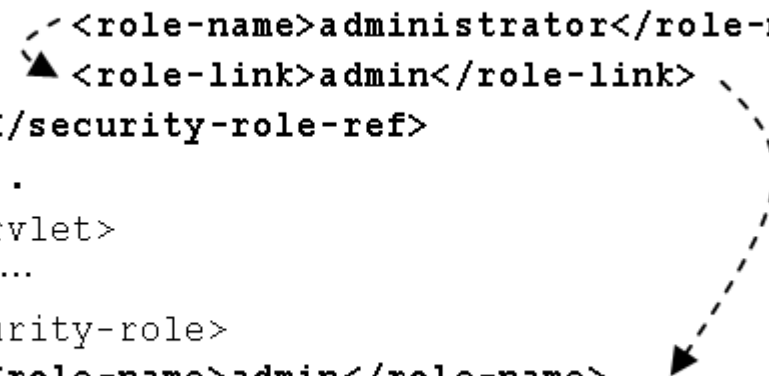
@WebServlet("/logout")
public class Logout extends HttpServlet {
    protected void doGet(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.logout();
        response.sendRedirect("login.html");
    }
}
```

程式安全管理

- 在Servlet 3.0之前，HttpServletRequest上就已存在三個與安全相關的方法：
getUserPrincipal()、
getRemoteUser() 及 **isUserInRole()**

```
if(request.isUserInRole("admin") || request.isUserInRole("manager")) {  
    // 進行站長或版面管理員才可以作的事，例如呼叫刪除討論串的方法之類的  
}
```

```
<web-app...>
  <servlet>
    <security-role-ref>
      <role-name>administrator</role-name>
      ▲ <role-link>admin</role-link>
    </security-role-ref>
    ..
  </servlet>
  // 略...
  <security-role>
    <role-name>admin</role-name>
    <role-name>manager</role-name>
  </security-role>
</web-app>
```



```
if(request.isUserInRole("administrator")) {
  // 略...
}
```


標註存取控制

```
...  
<security-constraint>  
  <web-resource-collection>  
    <web-resource-name>Admin</web-resource-name>  
    <url-pattern>/security</url-pattern>  
  </web-resource-collection>  
  <auth-constraint>  
    <role-name>admin</role-name>  
  </auth-constraint>  
</security-constraint>  
...  
  
@WebServlet("/admin")  
@ServletSecurity(  
    @HttpConstraint(rolesAllowed = {"admin"})  
)  
public class Admin extends HttpServlet {
```

標註存取控制

```
...  
<security-constraint>  
  <web-resource-collection>  
    <web-resource-name>Manager</web-resource-name>  
    <url-pattern>/security</url-pattern>  
    <http-method>GET</http-method>  
    <http-method>POST</http-method>  
    <http-method-omission>TRACE</http-method-omission>  
  </web-resource-collection>  
  <auth-constraint>  
    <role-name>admin</role-name>  
    <role-name>manager</role-name>  
  </auth-constraint>  
</security-constraint>  
...
```

標註存取控制

```
@WebServlet("/manager")
@ServletSecurity(
    value=@HttpConstraint(rolesAllowed = {"admin", "manager"}),
    httpMethodConstraints = {
        @HttpMethodConstraint(
            value = "GET", rolesAllowed = {"admin", "manager"}
        ),
        @HttpMethodConstraint(
            value = "POST", rolesAllowed = {"admin", "manager"}
        )
    }
)
public class Manager extends HttpServlet {
```

標註存取控制

- 在web.xml中設定

...

```
<login-config>
  <auth-method>FORM</auth-method>
  <form-login-config>
    <form-login-page>/login.html</form-login-page>
    <form-error-page>/error.html</form-error-page>
  </form-login-config>
</login-config>
```

...

標註存取控制

- 如果要設定<transport-guarantee>的對應資訊

```
...
@WebServlet(name="SecurityServlet", urlPatterns={"/security"})
@ServletSecurity(
    httpMethodConstraints = {
        @HttpMethodConstraint(
            value = "GET", rolesAllowed = {"admin", "manager"},
            transportGuarantee = TransportGuarantee.CONFIDENTIAL
        ),
        @HttpMethodConstraint(
            value = "POST", rolesAllowed = {"admin", "manager"},
            transportGuarantee = TransportGuarantee.CONFIDENTIAL
        )
    }
)
public class Security extends HttpServlet {
    ...
}
```

綜合練習 / 微網誌

- 將登入檢查、驗證等動作交給Web容器來負責